Fundamentos de Programação 2025-2026

Programming Fundamentals

Class #11 - Functional Programming Primitives

Overview

- Motivation
- Functional programming primitives
  - Map, filter, reduce
- Python Functional programming primitives
- Usage Examples
- Mistakes

Problem - Analyzing Student Grades

- Imagine you have a list of student grades for a class (maximum is 10):

  grades = [5, 7, 9, 3, 8, 9, 6, 10, 8]

- You want to:
  - Drop out failing grades (below 5).
  - Convert grades to percentages of the maximum grade.
  - Compute the class average (of normalized grades).

Possible imperative solution

```
grades = [5, 7, 9, 3, 8, 9, 6, 10, 8]

# Step 1: Filter
passing = []
for g in grades:
    if g >= 5:
        passing.append(g)

# Step 2: Map
normalized = []
for g in passing:
    normalized.append(g / 10 * 100)

# Step 3: Reduce
total = 0
for g in normalized:
    total += g
average = total / len(normalized)
print(average)
```

Comments

- This works …
- But:
  - it’s verbose
  - and requires managing state
    - passing, normalized, total.

Before we start our topic …

SOME USEFUL INFORMATION

Functional programming

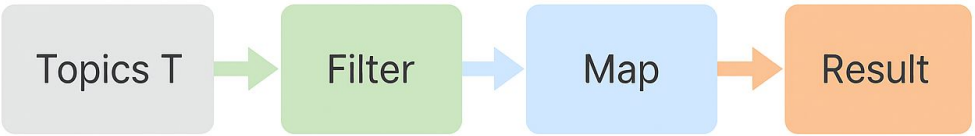
- Functional programming is a **declarative paradigm**
- Programs are built by **composing functions** that transform data.

Key Characteristics:

- **Declarative style**:
  - **Focus on *what* to do**, not *how* to do it
- **Pure functions**:
  - No side effects; output depends only on input
- **Immutability**:
  - Data is not modified — new values are returned
- **Function composition**:
  - Small functions are combined into pipelines

- Functional programming lets us process collections declaratively
  - efficiently

Filter – Map – Reduce

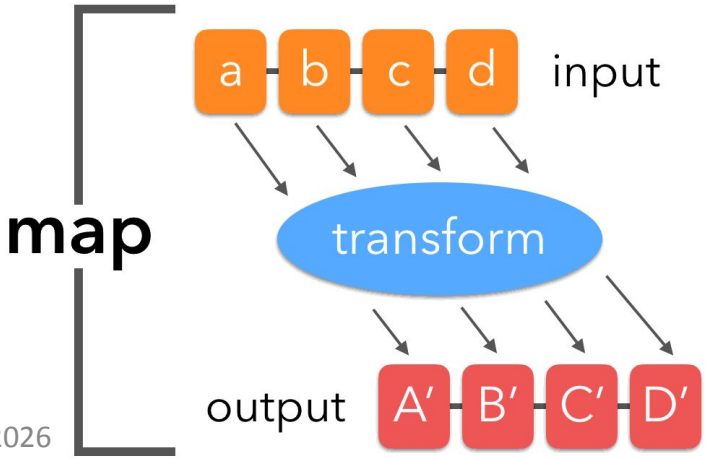


THREE FUNCTIONAL PROGRAMMING PRIMITIVES

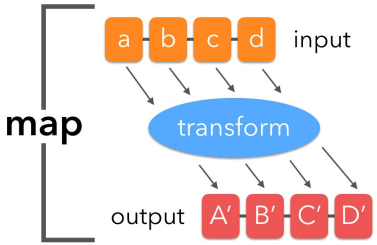
Map

- Map applies a transformation function to each element of a collection.
- Produces a new collection of the same size, with each element transformed.
- Think of it as:

Input Collection → Apply Function → Output Collection



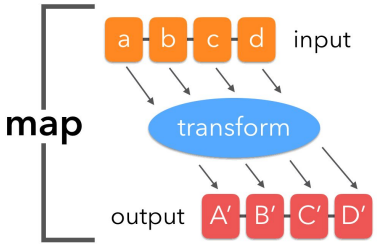
Map



Visual Metaphor

- Imagine a **conveyor belt**:
  - Items (elements) move along the belt.
  - At each station, a transformation is applied
    - e.g., paint, resize, relabel.
  - The belt delivers a new set of items, all transformed consistently.

Map

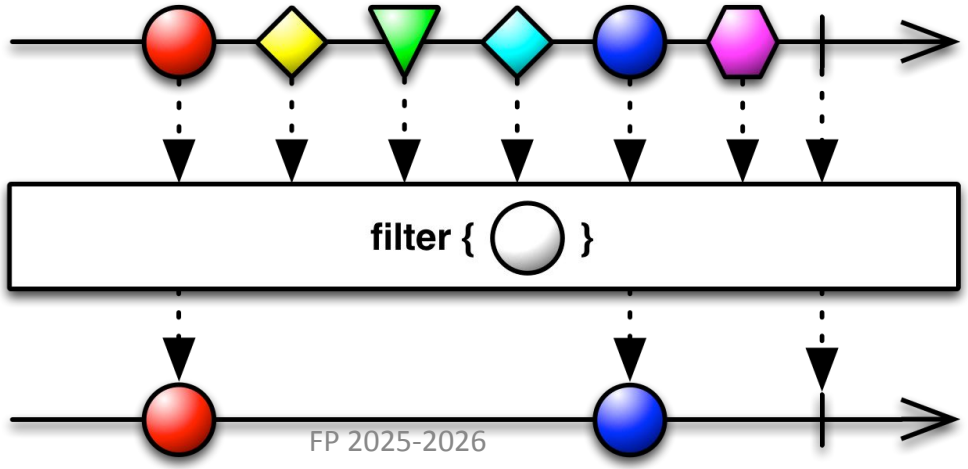


- Key Properties
  - Purely declarative
    - describes what transformation to apply, not how to loop.
  - Independent
    - each element is transformed separately.
  - Composable
    - can be chained with other primitives (e.g., filter, reduce).

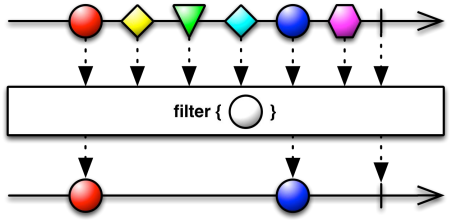
Filter: Selecting Elements Based on a Condition

- Filter takes a collection and a condition (predicate) and returns a new collection containing only the elements that satisfy the condition.
- Think of it as:

Input Collection → Apply Condition → Filtered Collection



Filter



Visual Metaphor

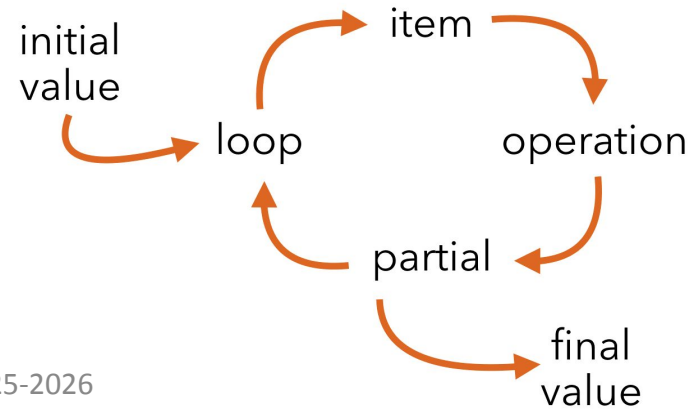
- Imagine a **sieve or quality control gate**:
  - Each item is inspected.
  - Only items that pass the test go through.
  - The rest are discarded.

- Typical Use Cases
  - Selecting valid data points
  - Filtering search results
  - Removing outliers or noise
  - Extracting relevant topics or entries

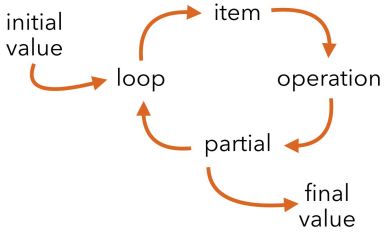
Reduce: Combining Elements into a Single Result

- Reduce takes a collection and a binary operation
- It combines elements one by one into a single accumulated result.
- Think of it as:

  Input Collection → Apply Combiner → Single Output



Reduce



Visual Metaphor

- Imagine a **compactor or folding machine**:
  - Items enter one by one.
  - Each is merged with the previous result.
  - The final output is a single, condensed value.

- Typical Use Cases
  - Summing values
  - Finding maximum or minimum
  - Merging strings or lists
  - Building a final result from parts

Map, Filter, Reduce

- **Filter** narrows the set.
- **Map** transforms the set.
- **Reduce** condenses the set into one result.

- Filter expresses selection without loops.
- Map expresses transformation without temporary variables.
- Reduce expresses aggregation without manual accumulation.

Flow (Pipeline)

Input —► Filter —► Map —► Reduce —► Result

- Filter
  - Select only the topics that meet a condition.
  - Example: keep only topics relevant to "computer science".
- Map
  - Transform each selected topic into a new representation.
  - Example: convert each topic into a short summary or keyword list.
- Reduce
  - Aggregate all transformed topics into a single outcome.
  - Example: combine summaries into one overall report or compute a relevance score.

Core Functional Primitives Beyond Map, Filter, Reduce

- Functional programming includes many other powerful primitives beyond map, filter, and reduce, such as:
  - fold,
  - zip, flatMap,
  - …

Fold (aka reduce with initial value)

- Generalized form of reduce that always starts with an initial accumulator.
- Useful for safely aggregating empty collections.

- Example: summing a list with initial value 0.

FlatMap

- Combines map and flatten.
- Applies a function that returns a collection, then flattens the result.
- Example: mapping each word to its synonyms, then flattening into one list.

Zip

- Combines two collections element-wise.

- Example: pairing names with scores
  $\rightarrow$ [(name1, score1), (name2, score2)].

Partition

- Splits a collection into two based on a predicate.

- Example: separating passing and failing grades.

Compose

- Combines two or more functions into one.

- Example: f(g(x)) becomes a single composed function.

Take / Drop / Slice

- Extract subsets of a collection.

- Examples:
  - **take** first 5 items
  - **drop** first 3
  - **slice** from index 2 to 6

Find / Exists / ForAll

- Query-style primitives
- **find**: returns first matching element.
- **exists**: checks if any element matches.
- **forall**: checks if all elements match.

Distinct / GroupBy / SortBy

- Data-cleaning and organizing tools
- **distinct**: remove duplicates.
- **groupBy**: cluster elements by key.
- **sortBy**: order elements by a property.

Why These Matter?

- They show expressiveness beyond loops and conditionals.

- They encourage modular thinking:
  - each function does one thing well.

- They support data pipelines:
  - chaining transformations without mutation.

IN PYTHON

Higher-level functions in Python

- Python provides a few higher order (or functional programming) functions in the standard library that can be quite useful:
  - map
  - filter
  - reduce*
  - lambda
  - list comprehensions

* Moved to a module in recent versions of Python

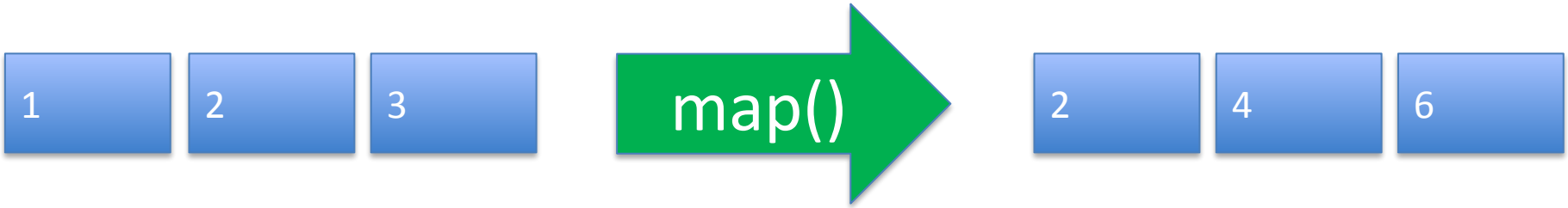
map()

- map() in python takes in a function and an iterable and applies the first to the second.

- It returns a map iterator, that applies the function in turn for every item it contains.
- This language choice was made to save memory and make loops slightly faster to process.

map()

- map(function, iterable)
- Example:
  list(map(lambda x: x*2, [1,2,3])) → [2, 4, 6]



- In python2 map() returns a list and in python3 map() returns a generator object.

filter()

- filter(function, iterable)
- Example:
  list(filter(lambda x: x%2!=0, [1,2,3])) → [1, 3]

- Similar to map(), in python2 filter() returns a list and in python3 filter() returns a generator object.

reduce()

- reduce(function, iterable)
- reduce() is no more a built-in function in Python ≥ 3.0
  - has been moved under functools.
- Example:

```
from functools import reduce
reduce(lambda a,b: a+b, [1,2,3]) → 6
```

Practical Examples

- map(): Cleaning text

  → `map(str.lower, words)`

- filter(): Removing invalid sensor readings

  → `filter(lambda x: x > 0, readings)`

- reduce(): Computing a checksum or total

  → `reduce(lambda x,y: x+y, values)`

Common Mistakes

- Forgetting to import reduce from functools
- Expecting map/filter to return lists automatically
- Using long lambda expressions (create named functions instead)
- Using reduce where sum() or built-ins are better

RETURNING TO OUR INITIAL PROBLEM …

Possible Functional Solution in Python

```python
from functools import reduce

grades = [5, 7, 9, 3, 8, 9, 6, 10, 8]

# Pipeline: Filter → Map → Reduce
passing_grades = filter(lambda g: g >= 5, grades)

normalized = map(lambda g: g / 10 * 100, passing_grades)

total = reduce(lambda acc, g: acc + g, normalized, 0)

count = len(list(filter(lambda g: g >= 5, grades)))

average = total / count if count > 0 else 0

print(average)
```

APPLICATION EXAMPLES

Additional Problem

- You have a dictionary of products and their prices:

```
orders = {
     "Laptop": 1200,
     "Mouse": 25,
     "Keyboard": 45,
     "Monitor": 300,
     "USB Cable": 8,
     "Headphones": 75
 }
```

- You want to:
  - **Filter** out cheap accessories (items under 20€).
  - **Map** the remaining prices to include tax (e.g., +23% VAT).
  - **Reduce** the taxed prices to compute the **total bill**.

Possible solution

```python
from functools import reduce

# Step 1: Filter out items under 20€
filtered_orders = dict(filter(lambda item: item[1] >= 20, orders.items()))

# Step 2: Map prices to include 23% VAT
taxed_orders = dict(map(lambda item: (item[0], item[1] * 1.23), filtered_orders.items()))

# Step 3: Reduce to compute total bill
total = reduce(lambda acc, price: acc + price, taxed_orders.values(), 0)

print("Filtered orders:", filtered_orders)
print("Taxed orders:", taxed_orders)
print("Total bill with VAT:", total)
```

Play